

Arrays, Multidimensional Matrices and Vectors in C++

📌 Definition to Remember

Arrays in C++ store contiguous collections of elements. Modern C++ provides `std::vector` for dynamic contiguous arrays with automated memory management and $O(1)$ random access.

1. Concept Overview & Fundamentals 🧠

Arrays, Multidimensional Matrices and Vectors in C++ provides modern high-performance capabilities combining low-level hardware control with high-level abstractions, object-oriented paradigms, and Standard Template Library (STL) utilities.

In modern C++ (C++11 through C++20):

- **Zero-Overhead Principle:** What you don't use, you don't pay for; what you do use, you couldn't hand code any better.
- **Type Safety & Streams:** Type-safe stream I/O (`std::cin` , `std::cout`) prevents runtime format mismatch vulnerabilities.
- **Resource Acquisition Is Initialization (RAII):** Automatic resource management via constructors, destructors, and smart pointers.

2. Technical Architecture & Memory Mechanics ⚙️

C++ Runtime Execution Architecture

```
+-----+
|                STACK SEGMENT                |
| - Local scope objects, references, RAII stack frames |
| - Deterministic automatic destruction upon scope exit |
+-----+
|
| v
| ^
|
+-----+
|                HEAP / FREE STORE              |
| - Dynamic objects allocated via new, std::vector, etc. |
| - Managed via delete / smart pointers          |
+-----+
|                STATIC & GLOBAL MEMORY         |
| - Global objects, static class variables, string literals |
+-----+
|                CODE / TEXT SEGMENT            |
| - Executable machine code, inlined functions, templates |
+-----+
```

Modern C++ Feature Comparison

Modern C++ Paradigm	Legacy Equivalent	Engineering Advantage	Performance Impact
<code>std::cin</code> / <code>std::cout</code>	<code>scanf</code> / <code>printf</code>	Type-safe, extensible for custom classes	Buffered I/O (optimizable via <code>ios::sync_with_stdio(0)</code>)
References (<code>Type&</code>)	Pointers (<code>Type*</code>)	Non-nullable, clean syntax without dereferencing	Direct alias; zero runtime overhead
<code>std::vector<T></code>	Raw Dynamic Array (<code>malloc</code>)	Automatic resizing, bounds safety, memory management	Amortized O(1) push_back, contiguous cache
<code>nullptr</code>	<code>NULL</code> / <code>0</code>	Strongly typed pointer literal preventing integer overload bugs	Zero overhead

3. Core Syntax & Implementation Patterns

Modern C++ Template Structure

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>

// Fast I/O and standard namespace configuration
int main() {
    std::ios_base::sync_with_stdio(false);
    std::cin.tie(NULL);

    std::cout << "Executing C++ Module: Arrays" << "\n";

    return 0;
}
```

4. Real-World Applications & System Engineering

- **Quantitative Finance & HFT:** High-frequency trading platforms use modern C++ for microsecond-level execution.
- **Game Engine Architecture:** Unreal Engine, Unity Core, and AAA game engines utilize C++ for real-time 3D rendering.
- **Deep Learning Frameworks:** Core tensor compute backends (PyTorch, TensorFlow, ONNX Runtime) are written in C++ and CUDA.
- **Web Browsers & Compilers:** Google Chrome (V8 engine), LLVM/Clang, and GCC are implemented in C++.

5. Comprehensive Solved Problems & Code Walkthroughs

Problem 1: array max min

Problem Description & Objective:

Implementation of array max min

C++ Code Implementation:

```

/*
 * Question:
 * Create a program to find the maximum and minimum element in an array.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Dynamic Size `std::vector<int>` & Linear Min/Max Search.
 *
 * 2. WHY IT IS USED?
 *    - `std::vector` manages dynamic array size safely.
 *
 * 3. HOW IT IS USED?
 *    - `if (arr[i] < minVal) minVal = arr[i]; if (arr[i] > maxVal) maxVal = arr[i];`
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Input size `n` and populate `vector<int> arr(n)`.
 * *   Step 2: Set `minVal = arr[0]` and `maxVal = arr[0]`.
 * *   Step 3: Loop `i` from 1 to `n-1`: update min/max.
 * *   Step 4: Output minimum and maximum.
 * =====
 */

#include <iostream>
#include <vector>
#include <algorithm>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int n;
    // Output formatted data to console stream
    cout << "Enter size of array: ";
    // Read user input from standard input stream
    cin >> n;

    // Evaluate conditional decision logic
    if (n <= 0) return 0;

    // Declare dynamic C++ std::vector container
    vector<int> arr(n);
    // Output formatted data to console stream
    cout << "Enter " << n << " elements: ";
    // Loop execution block
    for (int i = 0; i < n; i++) {
        // Read user input from standard input stream
        cin >> arr[i];
    }

    int minVal = arr[0];
    int maxVal = arr[0];

```

```
// Loop execution block
for (int i = 1; i < n; i++) {
    // Evaluate conditional decision logic
    if (arr[i] < minVal) minVal = arr[i];
    // Evaluate conditional decision logic
    if (arr[i] > maxVal) maxVal = arr[i];
}

// Output formatted data to console stream
cout << "Minimum element: " << minVal << endl;
// Output formatted data to console stream
cout << "Maximum element: " << maxVal << endl;

// Return zero code indicating successful program finish
return 0;
}
```

Problem 2: array occurrences

Problem Description & Objective:

Implementation of array occurrences

C++ Code Implementation:

```

/*
 * Question:
 * Create a program to find number of occurrences of an element in an array.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *   - Range-based For Loop (`for (int val : arr)`).
 *
 * 2. WHY IT IS USED?
 *   - Range-based for loops provide clean, safe element iteration without index out-of-bound errors.
 *
 * 3. HOW IT IS USED?
 *   - `for (int val : arr) if (val == target) count++;`
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Input vector elements and `target` value.
 * *   Step 2: Iterate range-based loop; increment `count` on match.
 * *   Step 3: Print occurrence count.
 * =====
 */

#include <iostream>
#include <vector>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int n, target, count = 0;

    // Output formatted data to console stream
    cout << "Enter array size: ";
    // Read user input from standard input stream
    cin >> n;

    // Declare dynamic C++ std::vector container
    vector<int> arr(n);
    // Output formatted data to console stream
    cout << "Enter elements: ";
    // Read user input from standard input stream
    for (int i = 0; i < n; i++) cin >> arr[i];

    // Output formatted data to console stream
    cout << "Enter target element to count: ";
    // Read user input from standard input stream
    cin >> target;

    // Loop execution block
    for (int val : arr) {
        // Evaluate conditional decision logic
        if (val == target) count++;
    }
}

```

```
// Output formatted data to console stream
cout << "Element " << target << " occurs " << count << " times." << endl;

// Return zero code indicating successful program finish
return 0;
}
```

Problem 3: array palindrome

Problem Description & Objective:

Implementation of array palindrome

C++ Code Implementation:

```

/*
 * Question:
 * Create a program to check if the array is palindrome or not.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Two-Pointer Technique (`start` and `end`).
 *
 * 2. WHY IT IS USED?
 *    - Checks symmetric elements from outer boundaries toward center in  $O(N/2)$  time.
 *
 * 3. HOW IT IS USED?
 *    - `while (start < end) { if (arr[start] != arr[end]) return false; start++; end--; }`
 *
 * 4. ALGORITHM EXPLANATION:
 *    * Step 1: Read vector elements.
 *    * Step 2: Initialize `start = 0`, `end = n - 1`.
 *    * Step 3: Compare `arr[start]` and `arr[end]`. If mismatch, set `isPalin = false` and break.
 *    * Step 4: Output Palindrome result.
 *    =====
 */

#include <iostream>
#include <vector>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int n;
    // Output formatted data to console stream
    cout << "Enter array size: ";
    // Read user input from standard input stream
    cin >> n;

    // Declare dynamic C++ std::vector container
    vector<int> arr(n);
    // Output formatted data to console stream
    cout << "Enter array elements: ";
    // Read user input from standard input stream
    for (int i = 0; i < n; i++) cin >> arr[i];

    bool isPalin = true;
    int start = 0, end = n - 1;
    // Loop execution block
    while (start < end) {
        // Evaluate conditional decision logic
        if (arr[start] != arr[end]) {
            isPalin = false;
            break;
        }
        start++;
    }
}

```

```
        end--;\n    }\n\n    // Output formatted data to console stream\n    if (isPalin) cout << "Array is a Palindrome." << endl;\n    // Output formatted data to console stream\n    else cout << "Array is NOT a Palindrome." << endl;\n\n    // Return zero code indicating successful program finish\n    return 0;\n}
```

Problem 4: array sorted check

Problem Description & Objective:

Implementation of array sorted check

C++ Code Implementation:

```

/*
 * Question:
 * Create a program to check if the given array is sorted.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Adjacent Element Comparison (`arr[i] > arr[i+1]`).
 *
 * 2. WHY IT IS USED?
 *    - Verifies non-decreasing order property.
 *
 * 3. HOW IT IS USED?
 *    - `for (int i=0; i<n-1; i++) if (arr[i] > arr[i+1]) isSorted = false;`
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Read vector elements.
 *   Step 2: Check adjacent pairs `arr[i] > arr[i+1]`.
 *   Step 3: If any pair breaks order, set `isSorted = false` and break.
 *   Step 4: Print Sorted status.
 * =====
 */

#include <iostream>
#include <vector>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int n;
    // Output formatted data to console stream
    cout << "Enter array size: ";
    // Read user input from standard input stream
    cin >> n;

    // Declare dynamic C++ std::vector container
    vector<int> arr(n);
    // Output formatted data to console stream
    cout << "Enter array elements: ";
    // Read user input from standard input stream
    for (int i = 0; i < n; i++) cin >> arr[i];

    bool isSorted = true;
    // Loop execution block
    for (int i = 0; i < n - 1; i++) {
        // Evaluate conditional decision logic
        if (arr[i] > arr[i + 1]) {
            isSorted = false;
            break;
        }
    }
}

```

```
// Output formatted data to console stream
if (isSorted) cout << "Array is Sorted in non-decreasing order." << endl;
// Output formatted data to console stream
else cout << "Array is NOT Sorted." << endl;

// Return zero code indicating successful program finish
return 0;
}
```

Problem 5: array sum average

Problem Description & Objective:

Implementation of array sum average

C++ Code Implementation:

```

/*
 * Question:
 * Create a program to find the sum and average of all elements in an array.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Accumulator Loop & Floating Average Division.
 *
 * 2. WHY IT IS USED?
 *    - Computes total and mean of array data.
 *
 * 3. HOW IT IS USED?
 *    - `sum += arr[i]; average = sum / n;`
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Input size `n`.
 *   Step 2: Accumulate `sum` during reading.
 *   Step 3: Divide `sum / n`.
 *   Step 4: Output sum and average.
 * =====
 */

#include <iostream>
#include <vector>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int n;
    // Output formatted data to console stream
    cout << "Enter size of array: ";
    // Read user input from standard input stream
    cin >> n;

    // Declare dynamic C++ std::vector container
    vector<int> arr(n);
    double sum = 0;

    // Output formatted data to console stream
    cout << "Enter " << n << " elements: ";
    // Loop execution block
    for (int i = 0; i < n; i++) {
        // Read user input from standard input stream
        cin >> arr[i];
        sum += arr[i];
    }

    double average = sum / n;

    // Output formatted data to console stream
    cout << "Sum: " << sum << endl;

```

```
// Output formatted data to console stream
cout << "Average: " << average << endl;

// Return zero code indicating successful program finish
return 0;
}
```

Problem 6: copy char array pointer

Problem Description & Objective:

Implementation of copy char array pointer

C++ Code Implementation:

```

/*
 * Question:
 * Write a function that uses pointer arithmetic to copy an array of char into another.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Pointer Arithmetic Traversal (`*dest = *src; src++; dest++;`).
 *
 * 2. WHY IT IS USED?
 *    - Direct pointer movement across contiguous memory blocks.
 *
 * 3. HOW IT IS USED?
 *    - `while (*src != '\0') { *dest = *src; src++; dest++; } *dest = '\0';`
 *
 * 4. ALGORITHM EXPLANATION:
 *    * Step 1: Pass `const char* src` and `char* dest`.
 *    * Step 2: Dereference `*src` and assign to `*dest`.
 *    * Step 3: Increment pointer addresses until null terminator `'\0'`.
 *    * Step 4: Null-terminate destination string.
 *    =====
 */

#include <iostream>

// Use standard library namespace globally
using namespace std;

void copyStringPointer(const char* src, char* dest) {
    // Loop execution block
    while (*src != '\0') {
        *dest = *src;
        src++;
        dest++;
    }
    *dest = '\0';
}

// --- Main Execution Entry Point ---
int main() {
    char source[] = "C++ Pointer String Copy";
    char destination[50];

    copyStringPointer(source, destination);

    // Output formatted data to console stream
    cout << "Source String: " << source << endl;
    // Output formatted data to console stream
    cout << "Copied Destination String: " << destination << endl;

    // Return zero code indicating successful program finish
    return 0;
}

```

Problem 7: delete array element

Problem Description & Objective:

Implementation of delete array element

C++ Code Implementation:

```

/*
 * Question:
 * Create a program to return a new array deleting a specific element.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *   - Filtering Vector Construction (`push_back`).
 *
 * 2. WHY IT IS USED?
 *   - Creates a dynamic vector containing only elements not equal to `elemToDelete`.
 *
 * 3. HOW IT IS USED?
 *   - `if (val != elemToDelete) newArr.push_back(val);`
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Input source vector and target element.
 * *   Step 2: Iterate elements; append non-matching elements to `newArr`.
 * *   Step 3: Print `newArr`.
 * =====
 */

#include <iostream>
#include <vector>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int n, elemToDelete;

    // Output formatted data to console stream
    cout << "Enter array size: ";
    // Read user input from standard input stream
    cin >> n;

    // Declare dynamic C++ std::vector container
    vector<int> arr(n);
    // Output formatted data to console stream
    cout << "Enter elements: ";
    // Read user input from standard input stream
    for (int i = 0; i < n; i++) cin >> arr[i];

    // Output formatted data to console stream
    cout << "Enter element value to delete: ";
    // Read user input from standard input stream
    cin >> elemToDelete;

    // Declare dynamic C++ std::vector container
    vector<int> newArr;
    // Loop execution block
    for (int val : arr) {
        // Evaluate conditional decision logic

```

```
        if (val != elemToDelete) {
            newArr.push_back(val);
        }
    }

    // Output formatted data to console stream
    cout << "New array after deletion: ";
    // Output formatted data to console stream
    for (int val : newArr) cout << val << " ";
    // Output formatted data to console stream
    cout << endl;

    // Return zero code indicating successful program finish
    return 0;
}
```

Problem 8: diagonal sum

Problem Description & Objective:

Implementation of diagonal sum

C++ Code Implementation:

```

/*
 * Question:
 * Create a program to find the sum of two diagonal elements.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - 2D Matrix Coordinate Access (`matrix[i][i]` and `matrix[i][n-1-i]`).
 *
 * 2. WHY IT IS USED?
 *    - Accesses primary diagonal `(i,i)` and secondary diagonal `(i, n-1-i)` in linear  $O(N)$  time.
 *
 * 3. HOW IT IS USED?
 *    - `primaryDiagonal += matrix[i][i]; secondaryDiagonal += matrix[i][n - 1 - i];`
 *
 * 4. ALGORITHM EXPLANATION:
 *    * Step 1: Input NxN matrix.
 *    * Step 2: Loop `i` from 0 to N-1: add `matrix[i][i]` to primary and `matrix[i][N-1-i]` to secondary.
 *    * Step 3: Output diagonal sums.
 *    =====
 */

#include <iostream>
#include <vector>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int n;
    // Output formatted data to console stream
    cout << "Enter dimension of square matrix (N x N): ";
    // Read user input from standard input stream
    cin >> n;

    // Declare dynamic C++ std::vector container
    vector<vector<int>> matrix(n, vector<int>(n));

    // Output formatted data to console stream
    cout << "Enter matrix elements row by row:" << endl;
    // Loop execution block
    for (int i = 0; i < n; i++) {
        // Loop execution block
        for (int j = 0; j < n; j++) {
            // Read user input from standard input stream
            cin >> matrix[i][j];
        }
    }

    int primaryDiagonal = 0;
    int secondaryDiagonal = 0;

    // Loop execution block

```

```
for (int i = 0; i < n; i++) {
    primaryDiagonal += matrix[i][i];
    secondaryDiagonal += matrix[i][n - 1 - i];
}

// Output formatted data to console stream
cout << "Primary diagonal sum: " << primaryDiagonal << endl;
// Output formatted data to console stream
cout << "Secondary diagonal sum: " << secondaryDiagonal << endl;

// Return zero code indicating successful program finish
return 0;
}
```

Problem 9: merge sorted arrays

Problem Description & Objective:

Implementation of merge sorted arrays

C++ Code Implementation:

```

/*
 * Question:
 * Create a program to merge two sorted arrays.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Two-Pointer Merge Algorithm.
 *
 * 2. WHY IT IS USED?
 *    - Merges two pre-sorted sequences into a single sorted output in O(N+M) time.
 *
 * 3. HOW IT IS USED?
 *    - `while (i < n1 && j < n2) if (a[i] <= b[j]) merged.push_back(a[i++]); else
merged.push_back(b[j++]);`
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Input two sorted vectors `a` and `b`.
 *   Step 2: Maintain indices `i` and `j`. Append smaller element to `merged`.
 *   Step 3: Append remaining elements from non-empty vector.
 *   Step 4: Output merged vector.
 * =====
 */

#include <iostream>
#include <vector>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int n1, n2;

    // Output formatted data to console stream
    cout << "Enter size of first sorted array: ";
    // Read user input from standard input stream
    cin >> n1;
    // Declare dynamic C++ std::vector container
    vector<int> a(n1);
    // Output formatted data to console stream
    cout << "Enter elements for first sorted array: ";
    // Read user input from standard input stream
    for (int i = 0; i < n1; i++) cin >> a[i];

    // Output formatted data to console stream
    cout << "Enter size of second sorted array: ";
    // Read user input from standard input stream
    cin >> n2;
    // Declare dynamic C++ std::vector container
    vector<int> b(n2);
    // Output formatted data to console stream
    cout << "Enter elements for second sorted array: ";
    // Read user input from standard input stream

```

```

for (int i = 0; i < n2; i++) cin >> b[i];

// Declare dynamic C++ std::vector container
vector<int> merged;
int i = 0, j = 0;

// Loop execution block
while (i < n1 && j < n2) {
    // Evaluate conditional decision logic
    if (a[i] <= b[j]) {
        merged.push_back(a[i++]);
    } // Alternative conditional branch
    else {
        merged.push_back(b[j++]);
    }
}

// Loop execution block
while (i < n1) merged.push_back(a[i++]);
// Loop execution block
while (j < n2) merged.push_back(b[j++]);

// Output formatted data to console stream
cout << "Merged sorted array: ";
// Output formatted data to console stream
for (int val : merged) cout << val << " ";
// Output formatted data to console stream
cout << endl;

// Return zero code indicating successful program finish
return 0;
}

```

Problem 10: reverse array

Problem Description & Objective:

Implementation of reverse array

C++ Code Implementation:

```

/*
 * Question:
 * Create a program to reverse an array.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - In-Place Array Reversal (`std::swap`).
 *
 * 2. WHY IT IS USED?
 *    - Reverses element order without requiring extra memory allocation.
 *
 * 3. HOW IT IS USED?
 *    - `swap(arr[start], arr[end]); start++; end--`;
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Read vector elements.
 *   Step 2: Set `start = 0`, `end = n - 1`.
 *   Step 3: Swap elements at `start` and `end`, move pointers inward.
 *   Step 4: Output reversed array.
 * =====
 */

#include <iostream>
#include <vector>
#include <algorithm>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int n;
    // Output formatted data to console stream
    cout << "Enter size of array: ";
    // Read user input from standard input stream
    cin >> n;

    // Declare dynamic C++ std::vector container
    vector<int> arr(n);
    // Output formatted data to console stream
    cout << "Enter elements: ";
    // Read user input from standard input stream
    for (int i = 0; i < n; i++) cin >> arr[i];

    int start = 0, end = n - 1;
    // Loop execution block
    while (start < end) {
        swap(arr[start], arr[end]);
        start++;
        end--;
    }

    // Output formatted data to console stream

```

```
cout << "Reversed Array: ";  
// Output formatted data to console stream  
for (int val : arr) cout << val << " ";  
// Output formatted data to console stream  
cout << endl;  
  
// Return zero code indicating successful program finish  
return 0;  
}
```

Problem 11: search 2d array

Problem Description & Objective:

Implementation of search 2d array

C++ Code Implementation:

```

/*
 * Question:
 * Create a program to search an element in a 2-D array.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - 2D Grid Traversal Search.
 *
 * 2. WHY IT IS USED?
 *    - Scans row by row `(i, j)` to locate target element.
 *
 * 3. HOW IT IS USED?
 *    - `if (matrix[i][j] == target) { found = true; break; }`
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Read 2D matrix dimensions and elements.
 *   Step 2: Read `target`.
 *   Step 3: Nested loops search for matching cell `(i,j)`.
 *   Step 4: Output row/col coordinates if found.
 * =====
 */

#include <iostream>
#include <vector>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int rows, cols, target;

    // Output formatted data to console stream
    cout << "Enter rows and columns of 2D array: ";
    // Read user input from standard input stream
    cin >> rows >> cols;

    // Declare dynamic C++ std::vector container
    vector<vector<int>> matrix(rows, vector<int>(cols));

    // Output formatted data to console stream
    cout << "Enter elements of " << rows << "x" << cols << " matrix:" << endl;
    // Loop execution block
    for (int i = 0; i < rows; i++) {
        // Loop execution block
        for (int j = 0; j < cols; j++) {
            // Read user input from standard input stream
            cin >> matrix[i][j];
        }
    }

    // Output formatted data to console stream
    cout << "Enter target element to search: ";

```

```

// Read user input from standard input stream
cin >> target;

bool found = false;
// Loop execution block
for (int i = 0; i < rows; i++) {
    // Loop execution block
    for (int j = 0; j < cols; j++) {
        // Evaluate conditional decision logic
        if (matrix[i][j] == target) {
            // Output formatted data to console stream
            cout << "Target " << target << " found at row " << i << ", col " << j << endl;
            found = true;
            break;
        }
    }
    // Evaluate conditional decision logic
    if (found) break;
}

// Output formatted data to console stream
if (!found) cout << "Target element not found in matrix." << endl;

// Return zero code indicating successful program finish
return 0;
}

```

Problem 12: sum average 2d array

Problem Description & Objective:

Implementation of sum average 2d array

C++ Code Implementation:

```

/*
 * Question:
 * Create a program to do sum and average of all elements in a 2-D array.
 */

/*
 * =====
 * CONCEPT EXPLORATION & ALGORITHM BREAKDOWN
 * =====
 * 1. WHICH CONCEPT IS USED?
 *    - Matrix Grid Accumulation.
 *
 * 2. WHY IT IS USED?
 *    - Sums all entries across rows and columns.
 *
 * 3. HOW IT IS USED?
 *    - `sum += matrix[i][j]; average = sum / (rows * cols);`
 *
 * 4. ALGORITHM EXPLANATION:
 * *   Step 1: Read dimensions `rows` and `cols`.
 *   Step 2: Accumulate `sum` during cell entry.
 *   Step 3: Divide by `rows * cols` to get average.
 *   Step 4: Print sum and average.
 * =====
 */

#include <iostream>
#include <vector>

// Use standard library namespace globally
using namespace std;

// --- Main Execution Entry Point ---
int main() {
    int rows, cols;

    // Output formatted data to console stream
    cout << "Enter rows and cols: ";
    // Read user input from standard input stream
    cin >> rows >> cols;

    // Declare dynamic C++ std::vector container
    vector<vector<int>> matrix(rows, vector<int>(cols));
    double sum = 0;

    // Output formatted data to console stream
    cout << "Enter matrix elements:" << endl;
    // Loop execution block
    for (int i = 0; i < rows; i++) {
        // Loop execution block
        for (int j = 0; j < cols; j++) {
            // Read user input from standard input stream
            cin >> matrix[i][j];
            sum += matrix[i][j];
        }
    }
}

```

```
int totalElements = rows * cols;
double average = sum / totalElements;

// Output formatted data to console stream
cout << "Sum of all 2D array elements: " << sum << endl;
// Output formatted data to console stream
cout << "Average of 2D array elements: " << average << endl;

// Return zero code indicating successful program finish
return 0;
}
```

6. Exam Scoring Points & Memory Revision

★ Must-Write Points

1. `std::vector` automatically manages heap buffer reallocation as elements are appended.
2. `std::vector::push_back()` has amortized $O(1)$ time complexity.
3. Two-pointer technique optimizes array reversal, palindrome verification, and sorted array merging.
4. Matrix operations: 2D vector representation (`vector<>`) simplifies dynamic grid processing.
5. Spatial locality: Contiguous layout maximizes CPU cache hit ratios for sequential traversals.

⚡ Quick Recall

Contiguous Buffer -> $O(1)$ Random Access -> `std::vector` Dynamic Growth -> Two-Pointer -> Cache Friendly